تفاعل الكائنات (Object Interaction)

1 الدالة الصديقة — Friend Function

ما هي؟

بشكل طبيعي، لا يمكن لأي دالة خارج الكلاس أن تصل لبياناته الخاصة (`private`). لكن في بعض الأحيان نحتاج دالة **مشتركة بين كلاسَيْن** — هنا يأتي دور الدالة الصديقة. نُعرّفها بالكلمة `friend` داخل الكلاس، وتُكتب بشكل طبيعي خارجه

```cpp
class sample
{
    int a, b;
public:
    void setvalue() { a=25; b=40; }
    friend float mean(sample s);  // إعلان الصداقة
};

// بدون — التعريف خارج الكلاس friend وبدون ::
float mean(sample s)
{
    return float(s.a + s.b) / 2.0;  // تصل لـ a و b رغم أنها private
}
```

المخرج: `mean value = 32.5`

خصائص الدالة الصديقة الأربع — احفظها كلها لأنها تأتي كـ True/False

الخاصية	التفصيل
ليست عضواً في الكلاس	تُعرَّف خارجه كدالة عادية
لا تُستدعى بالنقطة	`mean(x)` ✅ وليس `x.mean()` ❌
لا تستخدم `::` في تعريفها	بخلاف دوال الأعضاء العادية
يمكنها أن تكون صديقة لأكثر من كلاس	وهذا هو سبب وجودها أصلاً

> ⚠️ فخ امتحاني: الكلمة `friend` تُكتب فقط في داخل الكلاس (في الإعلان)، وتُحذف تماماً عند التعريف خارجه.

الدالة الصديقة لكلاسَيْن في نفس الوقت

عندما تريد دالة واحدة تعمل على كلاسَيْن مختلفَيْن، تُعلَن `friend` في كلا الكلاسَيْن. لاحظ أننا نحتاج **إعلاناً مسبقاً (Forward Declaration)** للكلاس الثاني:

```cpp
class abc;  // إعلان مسبق — ضروري جداً

class xyz
{
    int x;
public:
    friend void max(xyz, abc);  // صديقة للكلاسَيْن
};

class abc
{
    int a;
public:
    friend void max(xyz, abc);  // نفس الإعلان هنا أيضاً
};

void max(xyz m, abc n)          // التعريف بدون friend وبدون ::
{
    if(m.x >= n.a)
        cout << m.x;
    else
        cout << n.a;
}
```

> ⚠️ فخ امتحاني: لو حذفت `class abc;` من السطر الأول — الكود لا يُصرَّف (Compile Error)، لأن `xyz` لا تعرف شيئاً عن `abc` بعد.

سؤال MCQ متوقع:

> أيٌّ مما يلي صحيح بخصوص الدالة الصديقة؟
>
> - أ) تستخدم `::` عند تعريفها
> - ب) تُستدعى باستخدام كائن ونقطة
> - ج) ✅ يمكن إعلانها صديقة لأكثر من كلاس
> - د) يمكنها الوصول للأعضاء العامة فقط

❷ إرجاع الكائنات — Returning Objects

دالة في C++ يمكنها أن **تُرجع كائناً كاملاً** كما ترجع أي قيمة عادية. الكائن المُرجَع يكون **نسخة جديدة** تُعيَّن للمتغير في السطر الذي استدعى الدالة

```cpp
class complex
{
    float x, y;  // الجزء الحقيقي والتخيلي
public:
    void input(float real, float imag) { x=real; y=imag; }
    friend complex sum(complex, complex);
    void show(complex c) { cout << c.x << " +j " << c.y << endl; }
};

// الدالة ترجع كائناً من نوع complex
complex sum(complex c1, complex c2)
{
    complex c3;          // كائن مؤقت
    c3.x = c1.x + c2.x;
    c3.y = c1.y + c2.y;
    return c3;           // إرجاع الكائن كاملاً
}

int main()
{
    complex a, b, c;
    a.input(3.1, 5.65);
    b.input(2.75, 1.2);
    c = sum(a, b);       // c يستقبل الكائن المُرجَع
    c.show(c);
}
```

المخرج: `c = 5.85 +j 6.85`

> 📌 نقطة مهمة: نوع الإرجاع في رأس الدالة يكون اسم الكلاس `complex sum(...)` وليس `void`.

❸ المؤشر لأعضاء الكلاس — Pointer to Members

هذا الموضوع خاص بـ C++ ولا يوجد في C. الفكرة: بدلاً من الوصول لعضو بالاسم المباشر، نصنع **مؤشراً يشير لذلك العضو**

الصيغة الأساسية:

```text
نوع_البيانات  اسم_الكلاس :: * اسم_المؤشر = & اسم_الكلاس :: اسم_العضو
```

```cpp
class A
{
public:
    int m;
    void show();
};

// مؤشر يشير لعضو البيانات m
int A::*ip = &A::m;

// مؤشر يشير لدالة العضو show
void (A::*pf)() = &A::show;
```

طريقتان للوصول عبر المؤشر — الفرق بينهما سؤال تنافسي كلاسيكي

الحالة	المؤشر المستخدم	الصيغة	مثال
عندك **كائن** مباشر	`*.`	`object .* pointer`	`a .* ip`
عندك **مؤشر لكائن**	`*<-`	`ptr ->* pointer`	`ap ->* ip`

```cpp
A a;            // كائن مباشر
A *ap = &a;     // مؤشر لكائن

// كلا السطرين يطبعان نفس الشيء:
cout << a.*ip;     // استخدام *. مع الكائن المباشر
cout << ap->*ip;   // استخدام *<- مع المؤشر للكائن
```

⚠️ فخ امتحاني: هذا السطر خطأ حتى لو بدا منطقياً:

```cpp
int *ip = &m;  // ❌ خطأ — m وحده بدون اسم الكلاس لا معنى له
```

الصحيح دائماً:

```cpp
int A::*ip = &A::m;  // ✅ يجب استخدام :: مع الكلاس
```

تمرين Tracing — ما المخرج؟

```cpp
class M
{
    int x, y;
public:
    void set_xy(int a, int b) { x=a; y=b; }
    friend int sum(M);
};

int sum(M m)
{
    int M::*px = &M::x;
    int M::*py = &M::y;
    M *pm = &m;
    return m.*px + pm->*py;  // x + y
}

int main()
{
    M n;
    void (M::*pf)(int,int) = &M::set_xy;  // مؤشر لدالة
    (n.*pf)(10, 20);     // يستدعي set_xy(10,20)
    cout << sum(n);      // سطر 1

    M *op = &n;
    (op->*pf)(30, 40);   // يستدعي set_xy(30,40)
    cout << sum(n);      // سطر 2
}
```

السطر	القيم	المخرج
سطر 1	`x=10, y=20`	30
سطر 2	`x=30, y=40`	70

📊 جدول المقارنة الشامل — ملخص الوحدة

العنصر	Friend Function	Returning Object	Pointer to Member
الغرض	وصول لـ private من خارج الكلاس	إرجاع كائن كامل من دالة	الوصول لعضو عبر مؤشر
الكلمة المفتاحية	`friend`	اسم الكلاس كنوع إرجاع	`::*` و `.*` أو `->*`
تُعلَن في	داخل الكلاس	رأس الدالة	داخل الدالة أو الكلاس
تُعرَّف في	خارج الكلاس (بدون `::`)	خارج الكلاس (مع `::`)	حيثما تحتاجه
أهمية في MCQ	🔴 عالية جداً	🟡 متوسطة	🟡 متوسطة

🎯 أسئلة MCQ الأكثر توقعاً

سؤال (1):

> ما الفرق بين المؤشرَيْن `.*` و `->*`؟
> - أ) لا فرق بينهما
> - ب) ✅ `.*` مع كائن مباشر، و `->*` مع مؤشر لكائن
> - ج) `.*` للبيانات، و `->*` للدوال فقط
> - د) `->*` أسرع من `.*`

سؤال (2):

> أين تُكتب كلمة `friend` عند تعريف الدالة الصديقة خارج الكلاس؟
> - أ) قبل نوع الإرجاع
> - ب) بعد اسم الدالة
> - ج) ✅ لا تُكتب في التعريف — فقط في الإعلان داخل الكلاس
> - د) قبل وبعد اسم الكلاس

إضافة للدالة الصديقة — التمرير كمرجع (Pass by Reference):

تُمرر الكائنات كمرجع `&` داخل الدالة الصديقة عند الحاجة لتعديل قيمها الأصلية.

1. **تطبيق التبديل (Swapping):**

++C

```
void exchange (class-1 &x, class-2 &y)
{
    int temp = x.value1;
    x.value1 = y.value2;
    y.value2 = temp;
}
```

2. **تطبيق تحويل الرصيد (Bank Transfer):**

++C

```
int transfer (account2 &acc2, account1 &acc1, int amount)
{
    if(amount <= acc1.balance)
    {
        acc2.balance += amount;
        acc1.balance -= amount;
    }
}
```

إضافة لجدول المقارنة:

العنصر	Friend Function	Returning Object	Pointer to Member
تعديل الكائن	يتطلب استخدام `&` (Pass by Reference)	يُرجع نسخة جديدة من الكائن	يُعدل مباشرة عبر المؤشر

Export to Sheets